

Week 2 • Day 1

Feature Engineering — Part 1

Created 15 new predictive features from raw data to boost machine learning model performance

DATE	INTERN NAME	PROJECT	MENTOR
Day 8 of 28 Week 2	_____	Customer Churn Prediction & LTV Engine	_____

■ Technologies Used Today

Python

Pandas

NumPy

PostgreSQL

SQLAlchemy

■ Today's Key Metrics

16

New Features Created

37

Total Columns Now

7,043

Rows Preserved

5

Feature Categories

■ Day Objectives

- Understand feature engineering and why it matters for ML
- Create ratio features: charge_per_tenure, effective_tenure_paid
- Create service count and bundle features
- Create loyalty segment and contract risk score features
- Save engineered feature dataset to data/processed/

■ Tasks Completed Today

1. Created src/features/engineering.py with create_all_features() function
2. Feature 1: charge_per_tenure = MonthlyCharges / tenure (avoided division by zero for tenure=0)

3. Feature 2: $\text{effective_tenure_paid} = \text{TotalCharges} / \text{MonthlyCharges}$
4. Feature 3: $\text{revenue_potential} = \text{tenure} \times \text{MonthlyCharges}$
5. Features 4-9: binary flags for each add-on service (`_flag` suffix)
6. Feature 10: $\text{service_count} = \text{sum of all 6 add-on service flags}$
7. Feature 11: $\text{has_premium_services} = 1$ if $\text{service_count} \geq 3$
8. Feature 12: $\text{has_bundle} = \text{PhoneService AND InternetService together}$
9. Feature 13: loyalty_segment — 5 tiers based on tenure (0-12, 1-2yr, 2-4yr, 4-6yr, 6yr+)
10. Feature 14: contract_risk — 3=month-to-month, 2=one year, 1=two year
11. Feature 15: $\text{is_electronic_check}$ — flag for highest-churn payment method
12. Feature 16: is_auto_payment — flag for bank transfer or credit card auto
13. Ran script: confirmed new shape (7043, 37) — 16 new features added
14. Saved to `data/processed/telco_features.csv`
15. Committed to GitHub

■ Concepts Learned

- Feature engineering: creating new variables that capture patterns models can't see in raw data
- Ratio features: dividing two related columns reveals rate-of-change patterns
- `np.where()` for conditional feature creation without division by zero
- `pd.cut()` for creating ordinal bins (loyalty segments from continuous tenure)
- Why service count is more powerful than individual service flags for churn
- Business intuition drives feature engineering: 'What would a human analyst look at?'

■ Tools / Software Used

Tool / Library	Version / Notes	Status
NumPy	1.26.2	✓ In use — <code>np.where</code> for conditional logic
Pandas	2.1.4	✓ In use — <code>pd.cut</code> for binning

■ Outputs / Deliverables Produced

- ◆ `src/features/engineering.py` — feature creation module
- ◆ `data/processed/telco_features.csv` — 7,043 rows, 37 columns (21 original + 16 new)
- ◆ 16 new features documented with business rationale
- ◆ GitHub commit with engineering module and processed CSV

■ Problems Encountered & Solutions

Problem Faced	How I Solved It
---------------	-----------------

ZeroDivisionError for customers with tenure=0	Used np.where(df['tenure'] > 0, MonthlyCharges/tenure, MonthlyCharges) to handle tenure=0
pd.cut created NaN for tenure=0 with include_lowest=True missing	Added include_lowest=True parameter to pd.cut() to include the 0 value in the first bin

■ GitHub Commit Record

Commit Message	Week2-Day1: Feature engineering — 16 new features created, telco_features.csv saved
Branch	main
Files Changed	src/features/engineering.py, data/processed/telco_features.csv
Commit Hash	(fill after push)

■ End-of-Day Submission Checklist

■ engineering.py created	■ 16 new features created
■ No division by zero errors	■ Shape: (7043, 37) confirmed
■ telco_features.csv saved	■ charge_per_tenure created
■ service_count created	■ loyalty_segment created
■ contract_risk created	■ Day 8 commit on GitHub

■ Self Assessment

Week 2, Day 1			
9/10	6		8/28
Self Rating	Hours Spent	Progress	Day in Program

■ Key Learnings & Reflections

- ★ Feature engineering is where domain knowledge becomes a competitive advantage
- ★ np.where() is the Pandas-safe way to do conditional calculations (no ZeroDivisionError)

- ★ service_count is likely a strong predictor — customers invested in multiple services are more loyal
- ★ Always save engineered features to a new CSV — never overwrite the raw data

■ Plan for Tomorrow

- Apply One-Hot Encoding to multi-class categorical columns
- Scale numeric features with StandardScaler
- Create 80/20 stratified train/test split
- Save scaler.pkl and feature_columns.pkl to models/

INTERN SIGNATURE	MENTOR SIGNATURE	DATE SUBMITTED